

14주 차 보고서

(2026년 3월 29일 ~ 2026년 4월 4일)

작성자

박신우

이번 주 한 일

03.30 캐릭터 운전 애니메이션

기존 코드에서는 캐릭터가 트럭 앞좌석쪽에서 상호작용 키를 누르면, 트럭 운전은 되지만, 상호작용한 지점에 캐릭터가 멈추어 있고, 트럭을 운전하는 상태였다. 캐릭터가 운전석 위치에 앉아 운전을 하도록 하기 위해, 우선 믹사모에서 운전 애니메이션을 다운 받아 기존캐릭터에 애니메이션을 리타게팅 해주었다. 그 후 트럭에 탑승하면 운전석에 캐릭터를 부착해주는 함수를 만들어주었다.

```
void AFPSBaseCharacter::EnterTruckDriverSeat(ATruck* Truck)
{
    if (!Truck || bIsDrivingTruck || bIsOnTruckCargo || bIsUsingMountedWeapon)
    {
        return;
    }

    CurrentTruck = Truck;
    bIsDrivingTruck = true;
    bIsAiming = false;
    // 캐릭터가 다른 무언가에 붙어있으면 떼어버림
    DetachFromActor(FDetachmentTransformRules::KeepWorldTransform);
    // 운전석 위치에 캐릭터 부착
    AttachToComponent(Truck->DriverSeatPoint, FAttachmentTransformRules::SnapToTargetNotIncludingScale);
    // 회전 , 위치 0으로
    SetActorRelativeLocation(FVector::ZeroVector);
    SetActorRelativeRotation(FRotator::ZeroRotator);
    // 캐릭터 움직임 없앴
    if (GetCharacterMovement())
    {
        GetCharacterMovement()->StopMovementImmediately();
        GetCharacterMovement()->DisableMovement();
    }
    // 충돌처리 끄
    GetCapsuleComponent()->SetCollisionEnabled(ECollisionEnabled::NoCollision);
    GetMesh()->SetCollisionEnabled(ECollisionEnabled::NoCollision);

    PlayDrivingAnimation();
}
```

트럭 상호작용 함수에서, 운전석 탑승이 호출되면 EnterTruckDriverSeat 함수를 호출해 주었다.

```
AController* PlayerController = Character->GetController();
if (PlayerController)
{
    Character->EnterTruckDriverSeat(this);
    if (!Character->IsDrivingTruck())
    {
        return;
    }

    DriverCharacter = Character;
    PlayerController->Possess(this);
    PlayerController->SetControlRotation(GetActorRotation());
    UE_LOG(LogTemp, Log, TEXT("Driver Seat!"));
}
```

이번 주 한 일

그런데, 트럭에 탑승해보니 캐릭터가 운전석에 타는 것이 아니라 캐릭터 시작위치에서 운전을 하는 애니메이션이 호출되고 있었다.



원인을 찾아보니, 트럭에 탑승하는 순간

```
DriverCharacter = Character;  
PlayerController->Possess(this);
```

컨트롤 대상을 캐릭터에서 트럭으로 바꾼다.

그래서 이로 인하여 IsLocallyControlled가 false가 된다.

이로 인해 서버쪽 코드 처리로 인해 캐릭터 위치가 저렇게 된 것 같다.

그래서 트럭에 부착되어 있으면 남의 캐릭터인 경우의 코드가 작동 안하도록 해주었다.

```
if (IsLocallyControlled())  
{  
    MovePacketSendTimer += DeltaTime;  
    if (MovePacketSendTimer >= MOVE_PACKET_SEND_DELAY)  
    {  
        MovePacketSendTimer = 0.f;  
        SendMovePacket();  
    }  
}  
  
//(신우) 트럭을 타고 있을 때 위치보정 안하도록 (이거 안하니까 이상한곳에 앉아있음)  
else if (bIsAttachedToTruckSeat)  
{  
    return;  
}  
  
else // 남의 캐릭터인 경우
```

이번 주 한 일

이제 정상적으로 캐릭터가 운전석에서 운전하는 모습을 확인할 수 있다.



03.31 캐릭터 우클릭 줌인 줌 아웃

캐릭터가 우클릭을 하면 조금 더 정확하게 총기를 쏘도록 줌인 줌아웃 기능을 넣으려고 한다.

우선 마우스 우클릭을 하면 줌을 하도록 bindAction을 해주었다.

그 후 줌을 하면 bIsAiming이 true가 되도록 해주었고,
틱 함수에서 다음과 같이 코드를 입력했다.

조준중이면 카메라, 카메라 붐, 카메라 위치를 설정해둔 줌 값으로 바뀌게 해주었고,
보간을 사용해서 자연스럽게 카메라가 줌이 되도록 만들어 주었다.

```
// 줌 했을 때 FOV 확대
if (ThirdPersonCameraComponent && CameraBoom)
{
    // 조준중이면 Aiming 카메라, 아니면 기존
    const float TargetFOV = bIsAiming ? AimingThirdPersonFOV : DefaultThirdPersonFOV;
    // 이것도 조준중이면 Aiming 카메라 붐
    const float TargetBoomLength = bIsAiming ? AimingBoomLength : DefaultBoomLength;#
    // 이것도 조준중이면 Aiming 카메라 붐 소켓 오프셋
    const FVector TargetSocketOffset = bIsAiming ? AimingCameraBoomSocketOffset : DefaultCameraBoomSocketOffset;

    // 보간을 이용해서, 카메라가 부드럽게 이동하도록
    ThirdPersonCameraComponent->FieldOfView = FMath::FInterpTo(
        ThirdPersonCameraComponent->FieldOfView,
        TargetFOV,
        DeltaTime,
        AimInterpSpeed);

    CameraBoom->TargetArmLength = FMath::FInterpTo(
        CameraBoom->TargetArmLength,
        TargetBoomLength,
        DeltaTime,
        AimInterpSpeed);

    CameraBoom->SocketOffset = FMath::VInterpTo(
        CameraBoom->SocketOffset,
        TargetSocketOffset,
        DeltaTime,
        AimInterpSpeed);
}
```

이번 주 한 일

성공적으로 쏘이 되었는데, 한가지 문제가 발생하였다.

바로 크로스헤어 방향으로 총알이 날아가지 않는 문제이다.

아래 사진처럼 크로스 헤어 왼쪽 아래에 탄 이펙트가 생기는 것을 확인할 수 있다.



뷰포트 크기를 가져와서, 화면 중앙 좌표를 구한 후,

그 중앙 2D 중앙좌표값을 3D월드 좌표와 방향을 구해주었다.

```
// 실제 플레이어가 조종출일 때
if (const APlayerController* PlayerController = Cast<APlayerController>(Controller))
{
    // 뷰포트 크기 가져옴
    int32 ViewportSizeX = 0;
    int32 ViewportSizeY = 0;
    PlayerController->GetViewportSize(ViewportSizeX, ViewportSizeY);

    // 중앙 좌표 계산
    if (ViewportSizeX > 0 && ViewportSizeY > 0)
    {
        FVector WorldDirection;
        // 화면 중앙을 월드 방향으로 변환
        // 2D 화면 좌표를 3D 월드 좌표와 방향으로 바꾸어 주는 함수다
        if (PlayerController->DeprojectScreenPositionToWorld(
            ViewportSizeX * 0.5f,
            ViewportSizeY * 0.5f,
            OutLocation,
            WorldDirection))
        {
            OutRotation = WorldDirection.Rotation();
            return;
        }
    }
    // OutLocation -> 월드 공간에서 화면 중앙에 해당하는 위치
    // OutRotation -> 월드 공간에서 화면 중앙을 향하는 방향의 회전값
}
```

이걸 통해서, 플레이어의 크로스헤어가 어디를 가르키는지 구할 수 있게 되었다.

발사할 때 이 함수를 사용해서 크로스헤어 방향으로 총알이 잘 나가게 만들었다.

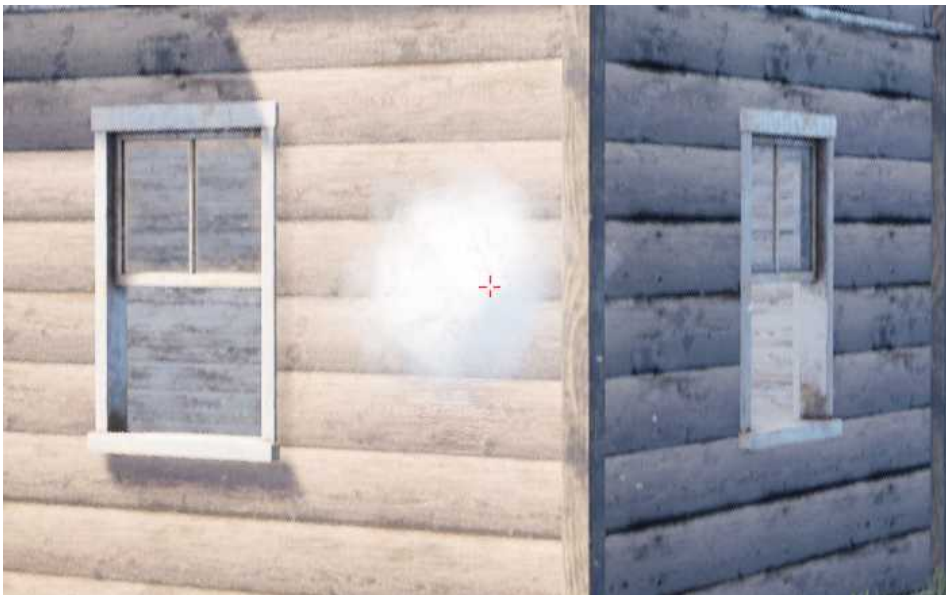
이번 주 한 일

또한 지금 현재 발사체를 총알이 잘 가나 확인하기 위해 메시 크기를 키우고, 속도도 줄인 상태였다.

속도를 키우고, 메시 크기도 줄여주었다.

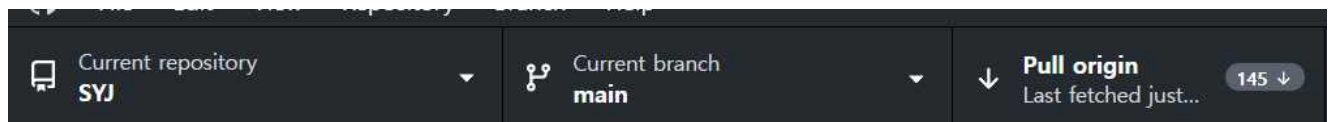
```
if (!ProjectileMovementComponent)
{
    // 이 컴포넌트를 이용해 발사체 이동 구현
    ProjectileMovementComponent = CreateDefaultSubobject<UProjectileMovementComponent>(TEXT("ProjectileMovementComponent"));
    ProjectileMovementComponent->SetUpdatedComponent(CollisionComponent);
    ProjectileMovementComponent->InitialSpeed = 20000.0f; // 초기 속도
    ProjectileMovementComponent->MaxSpeed = 20000.0f; // 최대 속도
    ProjectileMovementComponent->bForceSubStepping = true;
    ProjectileMovementComponent->bRotationFollowsVelocity = true; // 속도에 따라 회전
    ProjectileMovementComponent->bShouldBounce = false; // 바닥에 바운스
    ProjectileMovementComponent->Bounciness = 0.0f;
    ProjectileMovementComponent->ProjectileGravityScale = 0.0f; // 총알이 받는 중력
}
```

이제 크로스헤어 방향으로 나가는 것을 확인할 수 있었다.



03.31~ 04.01 병합문제

윤진이가 병합을 했는데, 잘되는지 봐달라고 연락이 왔다.



근데 깃풀이 145개가 있다고 뜨는 것이다.

저 때부터 매우 불안한 상태로 풀을 하였다.

설마 했는데, 역시나 월화 이틀 동안 한 작업물이 싹 다 날아간 상태였다.

그래도 브랜치를 판 상태로 작업을 해두었던 것이라, 다시 병합시키면 되지 않을까 하고 병합을 시켜봤는데 어림도 없었다.

주환이도 날아갔는지 물어봤는데, 주환이도 이틀동안 한 작업물들이 날아갔다고 하였다.

그래서 어떻게 고쳐야 할지 고민을 좀 하다가 셰이더 프로그래밍 시간에 이택희 교수님이 체리픽에 대해서 간단하게 설명을 해 주었던 것이 생각이 나서,

체리픽을 한 번 해보면 되지 않을까 생각이 들었다.

이번 주 한 일

그래서 날아간 커밋들의 커밋 해시를 찾아서 체리픽을 해주었다.

그랬더니 메인브랜치에서 날아갔던 작업들이 다시 정상적으로 작동되었다.

그래서 갑자기 자신감이 생겨서 주환이한테 전화를 걸어서 날아간 커밋들이 무엇인지 물어보고, 내가 고쳐보겠다고 말을 하였다.

그래서 주환이의 커밋들의 커밋 해시를 찾아서 체리픽을 시도하였는데,

잘 안되었다. 그래서 그냥 주환이 기숙사 방에 찾아가서 주환이와 함께 날아간 코드들을 복구하였다. 새벽까지 복구 하느라 너무 힘든 작업이었다.

나중에 알고 보니 주환이는 그 커밋들을 다른 브랜치에서 커밋한 것이 아니라,

메인에서 그냥 작업하여 메인에서 커밋을 한 것이었다.

main에서 직접 작업해서, 복구할 독립 커밋이나 안전한 원본 브랜치가 없어서 cherry-pick이 어려워졌던 것 같다.

그래도 정상적으로 복구를 성공하였다.

04.02 회의

04.03 기관총 애니메이션, 탄피 배출

우선 알파값들을 업데이트 해주고 UpdateFireAnimation 함수를 실행하는 함수를 만들었다.

```
void AMountedMachineGun::UpdateFireAnimationAlpha()
{
    // 알파값들 1로
    TriggerAnimationAlpha = 1.0f;
    RecoilAnimationAlpha = 1.0f;
    UpdateFireAnimation(0.0f);
}
```

UpdateFireAnimation 함수에서는 알파값들을 0으로 보간을 해주는 기능을 한다.

```
//알파값을 0으로 보간
if (DeltaTime > 0.0f)
{
    TriggerAnimationAlpha = FMath::FInterpTo(
        TriggerAnimationAlpha,
        0.0f,
        DeltaTime,
        TriggerAnimationReturnSpeed);

    RecoilAnimationAlpha = FMath::FInterpTo(
        RecoilAnimationAlpha,
        0.0f,
        DeltaTime,
        RecoilAnimationReturnSpeed);
}
```

이번 주 한 일

그 후 보간한 값을 애니메이션 블루프린트에 넣어준다.

```
// 보간한 값을 애니메이션 블루프린트에서 사용할 수 있도록 애니메이션 인스턴스의 프로퍼티로 설정
SetAnimFloatProperty(AnimInstance, TEXT("Anim_Trigger"), TriggerAnimationAlpha);
SetAnimFloatProperty(AnimInstance, TEXT("Trigger"), TriggerAnimationAlpha);
SetAnimVectorProperty(AnimInstance, TEXT("Gun_Translation"), GunRecoilTranslation * RecoilAnimationAlpha);
```

Anim_Trigger , Trigger 는 방아쇠 움직임용 값이고,
Gun_Translation는 총 몸체 반동 이동값이다.

기관총의 fire() 함수에서 UpdateFireAnimation()을 넣어주었더니 정상적으로 작동하였다.



그 후 아래와 같이 탄피를 생성해주는 함수를 만들어 줬다.
우선 소켓 위치를 얻은 후 그 위치에 탄피를 스폰해 주었다.

```
void AMountedMachineGun::SpawnEmptyShell()
{
    if (!EmptyShellClass || !GunMesh || !GetWorld() || !GunMesh->DoesSocketExist(EmptyShellSocketName))
    {
        return;
    }
    // 소켓 위치에서 스폰하도록 소켓 위치 가져와줌
    const FTransform SocketTransform = GunMesh->GetSocketTransform(EmptyShellSocketName, RTS_World);

    FActorSpawnParameters SpawnParams;
    SpawnParams.Owner = this;
    SpawnParams.Instigator = CurrentUser;
    SpawnParams.SpawnCollisionHandlingOverride = ESpawnActorCollisionHandlingMethod::AlwaysSpawn;

    // 탄피 스폰
    AActor* SpawnedShell = GetWorld()->SpawnActor<AActor>(
        EmptyShellClass,
        SocketTransform.GetLocation(),
        SocketTransform.Rotator(),
        SpawnParams);

    if (!SpawnedShell)
    {
        return;
    }
}
```

이번 주 한 일

그 후 탄피의 물리효과를 꺼주고 스폰한 다음, 다시 물리효과를 켜주었다.

멈춘 상태에서 위치를 잡고, 그 후에 물리엔진을 켜서 탄피가 떨어지게 해주었다.

```
ResetEmptyShellPhysics(SpawnedShell, false);  
// 탄피를 배출구 위치로 이동  
SpawnedShell->SetActorLocationAndRotation(  
    SocketTransform.GetLocation(),  
    SocketTransform.Rotator(),  
    false,  
    nullptr,  
    ETeleportType::TeleportPhysics);  
ResetEmptyShellPhysics(SpawnedShell, true);  
  
TArray<UPrimitiveComponent*> PrimitiveComponents;  
SpawnedShell->GetComponents<UPrimitiveComponent*>(PrimitiveComponents);  
  
const FVector WorldImpulse =  
    SocketTransform.GetRotation().RotateVector(EmptyShellEjectImpulse);
```

탄피가 잘 떨어지는 것을 확인할 수 있었다.



04.04 보고서 정리

다음 주 할 일

- 좀비 트럭으로 날려버리기
- 좀비 행동트리

특이사항

14주 차 보고서

(2026년 3 월 29 일 ~ 2026년 4 월 4 일)

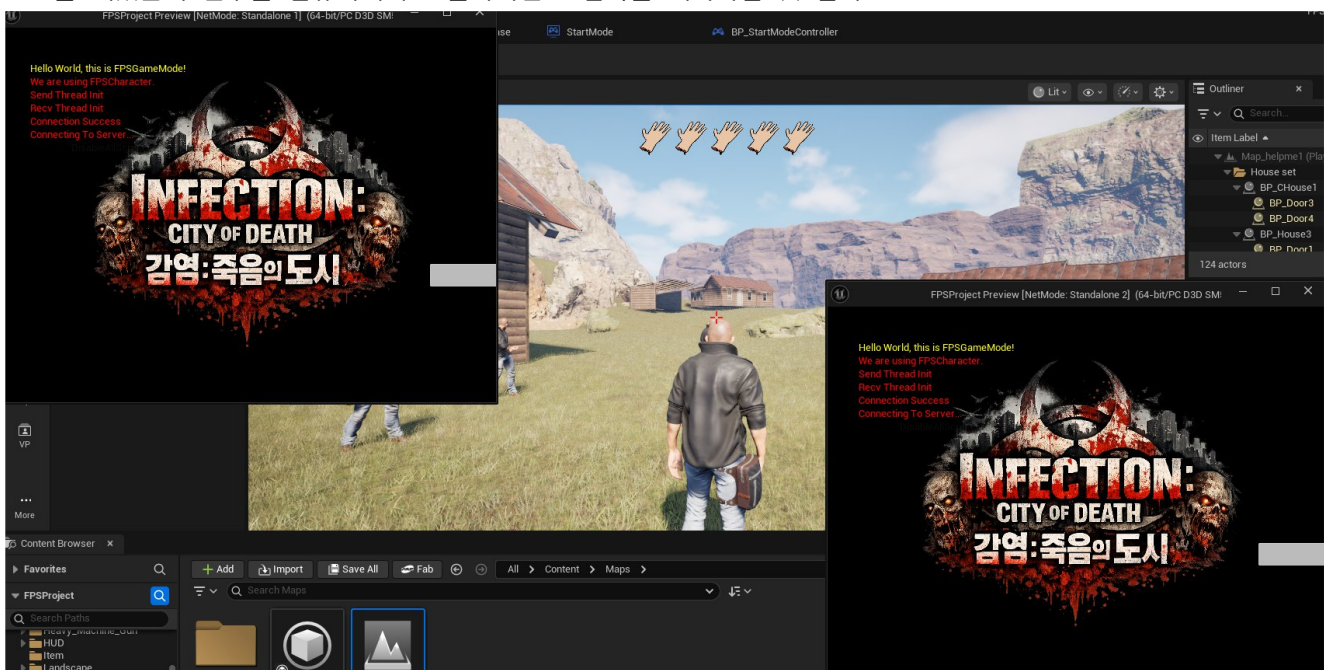
작성자

배주환

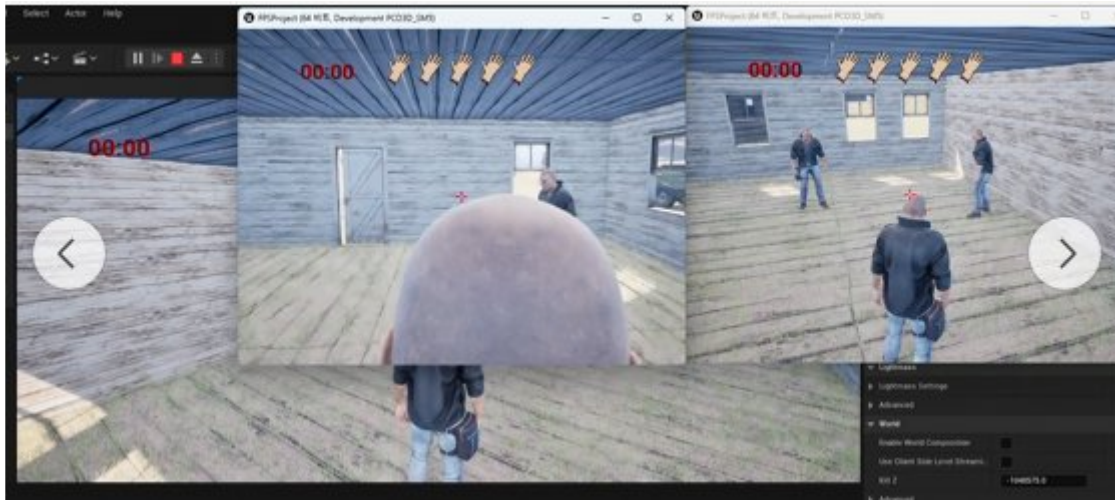
이번 주 회의 내용

이번 주 한 일

기존에 임시로 만들었던 로그인 UI 말고 클라이언트 친구가 만들어준 새 로그인 UI로 서버 연결을 시도하였다. 클라 입장에서는 로그인UI에 애니메이션이 들어가야 해서 로그인 맵을 따로 만들어주었는데 그게 내 입장에서는 너무 어려웠다. 언리얼의 각종 세부적인 설정을 바꾸기 / 기존에는 Ip주소 입력 -> 그 맵에서 연결했다면, 현재는 로그인 맵에서 새로운 맵으로 연결이 되면서 맵 로딩이 끝나기 전에 캐릭터가 스폰되는 문제가 발생하였다. 그래서 beginplay 이후로 서버의 응답을 받고 생성하는 걸로 바꾸었는데 또 다른 문제가 발생하였다. 기존에 멀티테스트 하던 방식으로 이제는 테스트를 하지 못하게 된것이다. 왜냐하면 기존에는 맵이 하나였기 때문에 하나의 프로세스에서 작동하여도 별 문제가 없었다. 하지만 현재는 맵이 두개가 되어서 run under one process라는 설정을 꺼야 했고 그러면 별도의 클라이언트로 인식을 하게 되어서 컴퓨터 자체에서 렉이 엄청 걸려서 캐릭터 동기화가 잘 되는지 등 디버깅을 하기가 어려워졌다. 일단은 테스트용으로 바로 연결이 되도록 만들것이다. 지금까지는 내 컴퓨터에서만 테스트를 해봤는데 친구들 컴퓨터에서도 클라이언트 접속을 해봐야할 것 같다.



기존에 연결하던 방식으로는 다음 맵으로 넘어가지 않는 모습이다.
예상치 못한 곳에서 막혀버려서 시간 낭비를 좀 한 것 같다.



이제 완전히 해결을 하고 캐릭터 동기화가 잘 되는 모습이다.

원래의 계획은 총을 쏘면 데미지가 적용이 되는 것까지 할 계획이었는데 팀원이 실수로 깃허브에 머지하다가 내 이전의 작업물을 다 지워버리는 사고가 있었어서 속도가 좀 늦어졌다.

다음 주 할 일

총에도 애니메이션이 있다고 해서 그것을 적용한 프로토콜을 만들어서 패킷을 보내고, 총알이 맞는 순간에 데미지가 적용이 되게 할 예정이다.

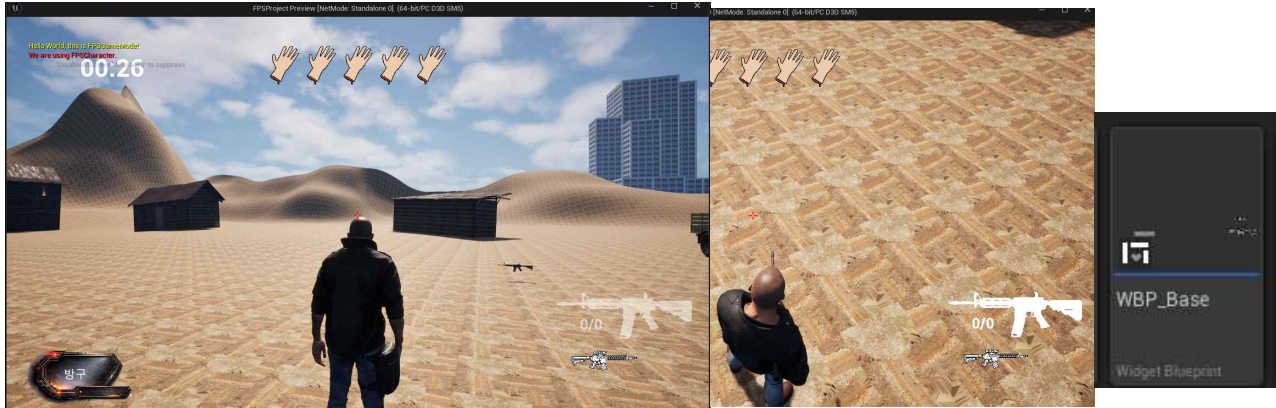
14주 차 보고서

(2026년 3 월 29 일 ~ 2026년 4 월 4 일)

작성자

안윤진

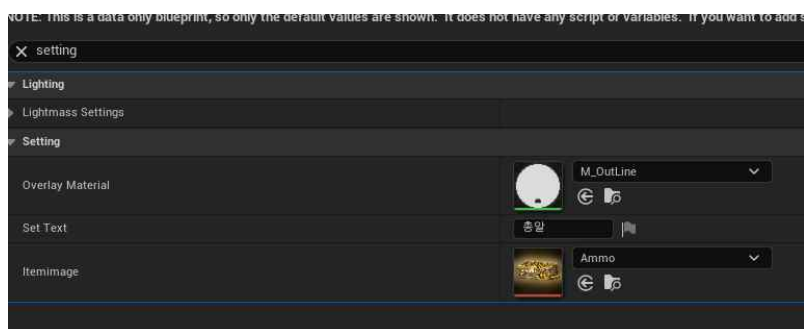
이번 주 한 일



총 ui추가 -> 총을 주우면 opacity 1로 변경을 통해 줍기 전후를 파악할 수 있다.

테스트로 inventory 위젯에 UI 배치해두었다.

위젯 베이스 제작, 후에 이 베이스로 총 UI, 닉네임ui 등 이동 후 ui 겹쳐 보이게 진행할 예정.

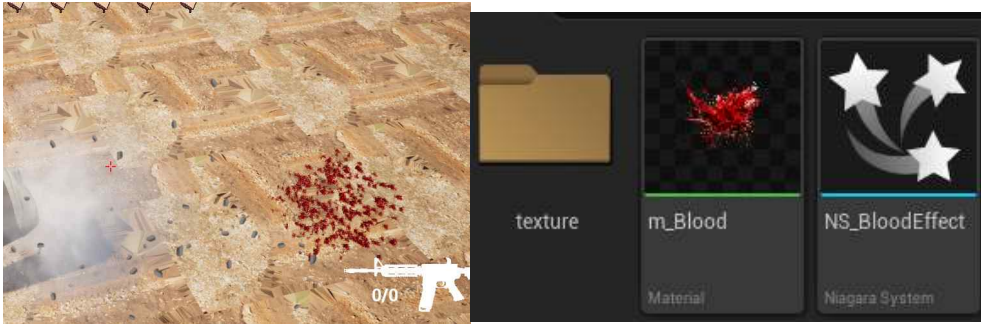


Item base class에 위젯과 텍스트 변수, 이미지 변수를 추가했다.

전에 인코딩 문제로 인해 한글을 쓰지 못했기에 텍스트 변수를 선언한 뒤 각 아이템bp에서 한글로 추가해두었다. 전에 제작해둔 상호작용 위젯에 이제 아이템 별로 각 이름이 뜬다.

또 아이템을 먹으면 그때마다 아이템에 맞는 이미지를 찾아서 슬롯 위젯을 변경하는 게 아닌, 아이템이 처음부터 이미지를 갖고있고 넘겨주는 방식으로 슬롯 이미지 변경을 해결했다.

이번 주 한 일



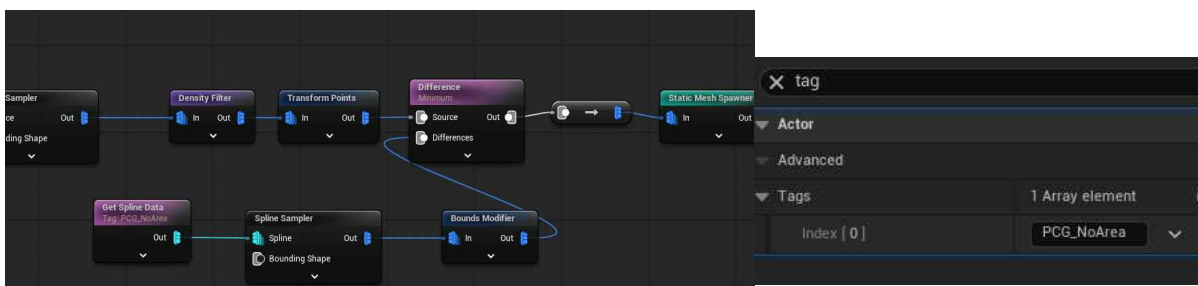
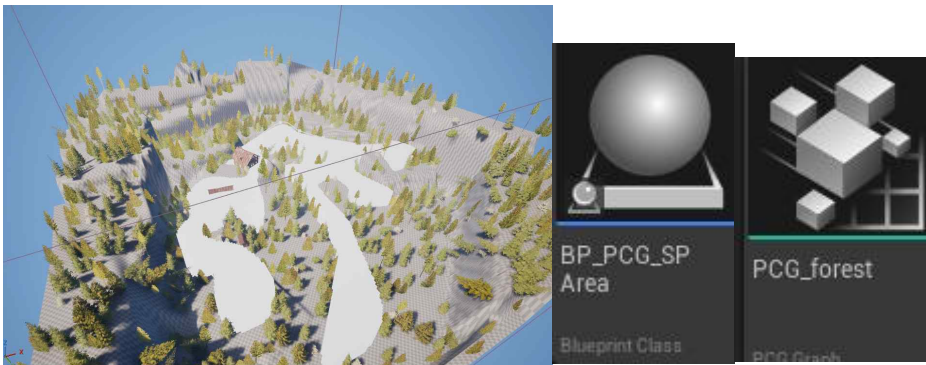
이제 피가 튀긴 한다. 좀비가 총에 맞고 피를 튀긴 이미지다. 순간 캡처가 어려워 이상해 보이지만, 기존에 신우형이 추가해둔 맞은 부위에 나타나는 이펙트를 내가 만든 것으로 변경하였다.

유튜버가 올려준 자료에서 피 텍스처를 얻어와 머티리얼을 제작하고 그것을 이용해 나이가아라로 제작하였다.

```
if (HitEffect)
{
    FVector Direction = -Hit.ImpactNormal;
    FRotator Rotation = Direction.Rotation();

    UNiagaraFunctionLibrary::SpawnSystemAtLocation(
        GetWorld(),
        HitEffect,
        EffectLocation,
        Rotation
    );
}
```

현재 피가 튀는 방향을 좀비base 코드에서 이렇게, 총을 쏜 플레이어 방향으로 튀도록 해두었는데 실제로 진행되는 건 약간 방향이 이상하다. 잘될때도 있고, 안될때도 있고.. 방향을 좀 더 자연스럽게 할 방법을 찾아야겠다. 또 지금은 딱 튀고 사라지는데, 팀원들과 회의 후 더 많은 양의 피가 튀었으면 좋겠다하여 변경할 예정이다.

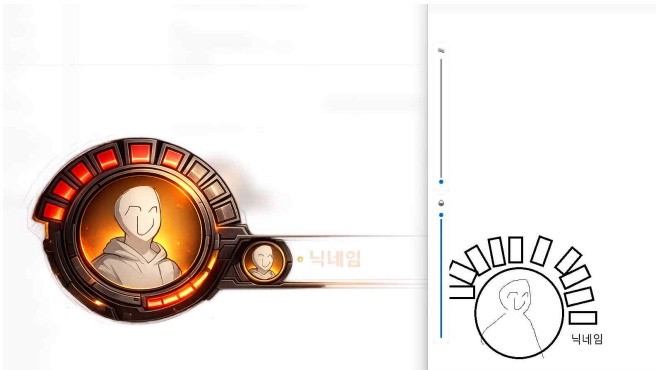


pcg 그래프로 나무 몇 종류들을 섞어 맵에 자동 배치하게 해줬다. 이때 스플라인 액터를 만들고, 이 액터에 태그를 통해 해당 태그 범위는 제외하고 나무가 생성되도록 하였다. 저 회색 부분들이 나무 생성 제외 범위이다. 나무가 옆으로 자라는 문제가 은근 많이 발생하는데 일단은 빠르게 제작해 본 뒤 수정하던가... 잘 보이지 않으면 그냥 넘어갈 것 같다.

이번 주 한 일



맵 퀄리티 이슈로 또 다시 만들었다. 주변 다른 팀의 맵도 좀 보고, 다른 괜찮은 맵들을 참고해 제작해보았다. 이번엔 크기를 더 키우고 랜드스케이프에 많은 공을 들였다... 전보단 나아 보인다.



그 외 지피티랑 ui제작하려고 많은 대화를 주고받았지만 막... 실제로 적용하긴 좀 애매하고, 괜찮아서 적용하면 위에 아이템 이미지마냥 깨져보이는 현상이 심해 사용하지 못했다.

다음 주 할 일

체력바 제작, 기관총 사용 시 ui 제작
연기(매연효과) 나이가가라 이펙트 제작

특이사항

정말 큰 사고를 쳤다. ifs문제로 깃pull이 안되어, 여러 명령어 사용해서 내 브랜치에선 해결하였는데... 머지에서 문제가 생겼다. 팀원들 커밋이 다 두 개씩 되고.. 내 며칠전 커밋이 팀원들 최근 커밋 위에 생기고... 날 아간거 살리느라 팀원들이 고생이 많았다. 다시 한 번 미안합니다.